

Customer360 - API Proxy (BFF): koncepcja architektury

└ Poziom: koncepcyjny. Szczegóły implementacyjne w kolejnych iteracjach.

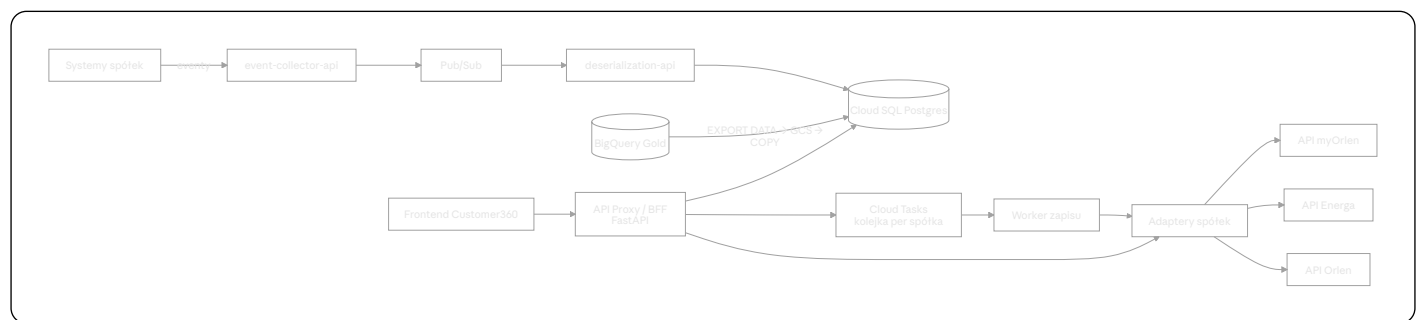
1. Kontekst

Model hybrydowy uzgodniony ze spółkami (myOrlen, Energa, Orlen):

- **Dane na żądanie** - spółki wystawiają REST API, a API Proxy odpytuje je w trakcie rozmowy operatora.
- **Eventy tylko tam, gdzie niezbędne** - np. profil nowo zarejestrowanego klienta przez `event-collector-api`, żeby nie czekać na batch D-1.
- **Dane referencyjne z Datalake** - kontakty, ID klientów w spółkach i mapowania ładowane cyklicznie z BigQuery.

API Proxy udostępnia frontendowi Customer360 **jeden spójny model danych** (jednolita paginacja, sortowanie, filtrowanie) i przyjmuje **żądania zmian**, które przekazuje do spółek.

2. Widok ogólny



3. Warstwa translacji (odczyt)

Wzorzec: API Composition + Anti-Corruption Layer, czyli adapter per spółka.

- **Model kanoniczny** - jeden model Customer360 (np. `CustomerBasic`, `Invoice`, `Consent`) i jedno kanoniczne zapytanie (filtry, sortowanie, paginacja).
- **Adapter per spółka** tłumaczy zapytanie kanoniczne na request do spółki, a odpowiedź spółki z powrotem na model kanoniczny. Adaptery pisane w kodzie (Pydantic), bez deklaracyjnych mapowań - przy 3 źródłach kod jest prostszy w utrzymaniu i debugowaniu.
- **Fan-out** - równoległe wywołania async do spółek, merge wyników w BFF.
- **Deklaracja możliwości** - każdy adapter deklaruje, które filtry i sortowania obsługuje. Filtrów nieobsługiwanych przez źródło nie dorabiamy lokalnie na stronicowanych danych, tylko jawnie zawężamy kontrakt.
- **Paginacja** - dane pojedynczego klienta pobieramy po ID, bez paginacji. Listy (faktury, transakcje) prezentujemy per spółka, każda z własną paginacją. Wspólną listę z trzech źródeł (cursor złożony) odkładamy, dopóki biznes jej wyraźnie nie potrzebuje.
- **Częściowe awarie** - timeout per adapter i odpowiedź z informacją, które źródło jest niedostępne. Operator dostaje np. 2/3 danych z ostrzeżeniem zamiast błędu.

- **Pierwszeństwo źródeł** – przy konflikcie danych z Postgresa (eventy, DLH) i danych z API spółki wygrywa nowszy znacznik czasu źródła.

4. Postgres jako baza BFF

Trzy role:

1. **Lookup klienta** – po telefonie, e-mailu i innych kluczach szukamy ID klienta w każdej spółce, co od razu daje kontekst „z kim rozmawia operator”.
2. **Dane przygotowane wstępnie** – kontakty, mapowania ID.
3. **Rejestr ruchu** – odczyty, zapisy, statusy zadań.

Model danych:

- **Staging (append-only, z timestampem)** – zasilany batchem z DLH i eventami rejestracji.
- **Tabela current** – zdeduplikowana, tylko aktualny stan klienta, czytana przez BFF.

5. Zasilanie z Datalake

Composer DAG → BigQuery (**EXPORT DATA**) do GCS → Cloud Run Job z (**COPY**) do Postgresa.

- Wszystkie komponenty już istnieją, więc bez nowych kosztów stałych. Eksport z BQ jest darmowy, (**COPY**) to najszybsza ścieżka ładowania.
- W dbt (Gold) powstaje jeden model dokładnie w kształcie tabeli ładowanej do Postgresa, więc logika zostaje w jednym miejscu.
- Batch ładuje do osobnej tabeli, a potem jedną krótką instrukcją przenosi dane do stagingu (patrz watermark niżej).

6. Przeliczenie staging → current

Jedna procedura inkrementalna uruchamiana przez (**pg_cron**) (co ~1 min), dodatkowo wywoływana przez Composer zaraz po batchu.

- Procedura bierze wiersze powyżej watermarku, wybiera najnowszy rekord per klient i robi upsert do current tylko wtedy, gdy rekord jest nowszy.
- **Watermark po sekwencji** (**bigserial**), **nie po timestampie**. Przetwarzamy tylko do ID, poniżej którego nie ma już otwartych transakcji.
- **Advisory lock**, żeby przebiegi się nie nakładały.
- **Usunięcia** – rekordy nieobecne w batchu i starsze niż moment odcięcia batcha usuwamy lub oznaczamy.
- **Retencja** – staging partycjonowany po dniu, stare partycje usuwane.
- **Bez triggerów** – spowolniłyby (**COPY**) i powiązały przyjmowanie eventów z logiką deduplikacji.
- **Monitoring** – (**cron.job_run_details**) i alert na nieudane przebiegi.

7. Żądania zmian (zapis asynchroniczny)

Wzorzec: Asynchronous Request-Reply. Postgres = źródło prawdy o zadaniu, Cloud Tasks =

dostarczenie.

1. **Przyjęcie** – zapis zadania w Postgresie (`ACCEPTED`), utworzenie taska w Cloud Tasks, operator dostaje `202` z ID zadania.
2. **Dispatch** – kolejka per spółka, worker wywołuje API spółki przez adapter, spółka zwraca swój numer referencyjny, status zmienia się na `SUBMITTED`.
3. **Zamknięcie** – spółka wysyła event przez `event-collector-api` (korelacja po ID zadania) albo, jeśli nie umie, jest odpytywana (polling). Status: `DONE` / `FAILED`.
4. **Operator** – `GET /tasks/{id}`, lista zadań klienta, w UI widoczne oznaczenie „zmiana w toku”.

Dlaczego Cloud Tasks zamiast Pub/Sub:

- Ruch punkt-punkt, a nie fan-out.
- Rate limit per spółka w konfiguracji.
- Deduplikacja po nazwie taska.
- Retry z backoffem.
- Możliwość pauzy kolejki na czas awarii spółki.

Sweeper (Cloud Scheduler, co kilka minut):

- ponownie wysyła zadania wiszące w `ACCEPTED`, co zastępuje outbox;
- odpytuje spółki o zadania w `SUBMITTED`;
- oznacza zadania po przekroczeniu SLA.

Dodatkowo: klucz idempotencji przekazywany do spółki, SLA per typ zadania, historia przejść statusów jako audyt.

8. Co warto dodać do architektury

- **Log dostępu do danych osobowych** – kto (który operator) oglądał którego klienta i kiedy. To wymóg RODO niezależny od rejestru ruchu technicznego.
- **Uwierzytelnianie i autoryzacja operatorów** – SSO organizacji. Uprawnienia per spółka i per zakres danych muszą być egzekwowane w BFF, nie tylko we frontendzie.
- **Correlation ID przez cały łańcuch** (frontend → BFF → adapter → spółka → zadanie → event zamknięcia) – bez tego diagnoza incydentów między systemami będzie bardzo trudna.
- **Kontrakt API spółek** – specyfikacja OpenAPI, wersjonowanie i testy kontraktowe adapterów. Zmiana po stronie spółki nie może być wykrywana dopiero przez operatora.
- **Mocki / środowiska testowe spółek** – do rozwoju i testów adapterów bez zależności od dostępności ich środowisk.
- **SLO per spółka** – czas odpowiedzi i dostępność. Podstawa do rozmów o jakości i do ustawienia timeoutów.
- **Wieloznaczność lookupu** – jeden numer telefonu może należeć do kilku klientów

(rodzina, firma). UI musi obsługiwać wybór, a nie zakładać jedno dopasowanie.

- **Krótki cache odpowiedzi spółek (TTL w sekundach/minutach)** – opcjonalnie, żeby ograniczyć powtórne odpytywanie w trakcie jednej rozmowy.
- **Rejestr ruchu a pojemność Postgresa** – przy dużym wolumenie rozważyć partycjonowanie i okresowy odpływ historii do BigQuery.

9. Pytania otwarte

Biznes / prawne

- Czy jest podstawa prawna i zgoda na wspólny wgląd w dane klienta w ramach grupy kapitałowej? Kto może widzieć dane której spółki?
- Jakie są wymagania retencji dla rejestru ruchu i logu dostępu?
- Które operacje zapisu są w zakresie pierwszej wersji (e-mail, telefon, adres, zgody)?
- Jakie SLA na zamknięcie zadania jest akceptowalne dla każdego typu zmiany? Kto reaguje na przekroczenie?

Spółki

- Czy wszystkie spółki udostępnią API w terminie i z jakim poziomem dostępności?
- Czy spółki potrafią wysłać event zakończenia zadania, czy potrzebny jest polling ich endpointu statusu?
- Jakie są limity obciążenia ich API (req/s) oraz okna serwisowe?
- Jak spółki identyfikują klienta w API i czy te identyfikatory są stabilne?
- Czy zapisy obsługują idempotencję (klucz przekazywany od nas)?

Techniczne / DLH

- Jaki jest wolumen bazy klientów do ładowania (miliony wierszy?) – potrzebne do wymiarowania Cloud SQL.
- Kto jest właścicielem jakości mapowania ID (identity resolution) – DLH czy spółki?
- Jak szybko nowa rejestracja musi być widoczna dla operatora (sekundy / minuta)?
- Wysoka dostępność Cloud SQL (HA) – czy wymagana dla operatorów pierwszej linii?